Smart Pointers in Modern C++

Theory, Edge Cases & Code Examples

- ✓ Smart Pointer Types and Ownership Models
- ✓ RAII and Automatic Resource Management
- ✓ Modern C++ Best Practices with Smart Pointers
- ✓ Circular References with shared_ptr
- ✓ Creating shared_ptr from unique_ptr

→ *Swipe to learn* →

Smart Pointers in Modern C++

Theory

📌 1. Smart Pointer Types and Ownership Models (1/4)

Smart pointers are C++11 template classes that provide automatic memory management through RAII (Resource Acquisition Is Initialization). They encapsulate raw pointers and automatically delete managed objects when no longer needed, eliminating manual memory management and preventing memory leaks, dangling pointers, and double deletion bugs.

The Three Smart Pointer Types

Type	Ownership Model	Copyable	Movable	Size (Typical)	Overhead	Primary Use Case
`std::unique_ptr`	Exclusive (single owner)	❌ No (deleted)	✅ Yes	8 bytes	Zero	Default choice for single ownership
`std::shared_ptr`	Shared (reference counted)	✅ Yes	✅ Yes	16 bytes	Atomic ops, control block	Multiple owners of same resource
`std::weak_ptr`	Non-owning observer	✅ Yes	✅ Yes	16 bytes	Minimal	Break cycles, observe shared objects

Smart Pointers in Modern C++

Theory

📌 1. Smart Pointer Types and Ownership Models (2/4)

Ownership Semantics Comparison

Aspect	unique_ptr	shared_ptr	weak_ptr
Ownership	Exactly one owner	Multiple owners allowed	Does not own (observes only)
Deletion Timing	When unique_ptr destroyed	When last shared_ptr destroyed	Never deletes (non-owning)
Copy Behavior	Deleted at compile time	Creates another owner (refcount++)	Observes without owning
Move Behavior	Transfers ownership, source becomes null	Creates new owner via control block	No ownership to transfer
Access	Direct via `*` or `->`	Direct via `*` or `->`	Must `lock()` to get temporary shared_ptr
Thread Safety	Not thread-safe (single owner)	Control block ops are atomic	Control block ops are atomic
Performance	Same as raw pointer	Atomic operations overhead	Minimal overhead
Memory Layout	Just pointer (+ deleter if custom)	Pointer + control block pointer	Pointer + control block pointer

Smart Pointers in Modern C++

Theory

📌 1. Smart Pointer Types and Ownership Models (3/4)

Code Example: Ownership Models

```
#include <memory>
#include <iostream>

void demonstrateOwnership() {
    // ✅ unique_ptr: Exclusive ownership
    std::unique_ptr<int> up1 = std::make_unique<int>(42);
    // auto up2 = up1;  // ❌ Compile error: copy deleted
    auto up2 = std::move(up1);  // ✅ Ownership transferred, up1 = nullptr

    // ✅ shared_ptr: Shared ownership
    std::shared_ptr<int> sp1 = std::make_shared<int>(100);
    auto sp2 = sp1;  // ✅ Both own the object
    auto sp3 = sp1;  // ✅ Now 3 owners (refcount = 3)
    std::cout << "Refcount: " << sp1.use_count() << "\n";  // 3

    // ✅ weak_ptr: Non-owning observation
    std::weak_ptr<int> wp = sp1;  // Observes, doesn't increment refcount
    std::cout << "Refcount after weak: " << sp1.use_count() << "\n";  // Still 3

    if (auto temp = wp.lock()) {  // ✅ Temporary shared_ptr for safe access
        std::cout << "Value: " << *temp << "\n";
    }
}
```

Smart Pointers in Modern C++

Theory

📌 1. Smart Pointer Types and Ownership Models (4/4)

Decision Tree: Which Smart Pointer to Use?

Question	Answer	Recommended Type
Do multiple parts of code need to share ownership?	No	`std::unique_ptr`
Do multiple parts of code need to share ownership?	Yes → Do you need to observe without owning?	No → `std::shared_ptr`
Do multiple parts of code need to share ownership?	Yes → Do you need to observe without owning?	Yes → `std::weak_ptr`
Managing arrays?	Yes → Fixed size known at compile time?	Yes → `std::array`
Managing arrays?	Yes → Fixed size known at compile time?	No → `std::vector`
Managing arrays?	Yes → Must use C-style array?	Yes → `std::unique_ptr<T[]>`
Managing non-memory resources (files, sockets)?	Yes	`unique_ptr` or `shared_ptr` with custom deleter
Interfacing with legacy API that returns raw pointer?	Yes	Wrap immediately in appropriate smart pointer

---

Smart Pointers in Modern C++

Theory

📌 2. RAII and Automatic Resource Management (1/4)

Smart pointers implement the RAII idiom by tying resource lifetime to object lifetime. Resources are acquired in constructors and automatically released in destructors, even when exceptions occur.

How Smart Pointers Implement RAII

RAII Principle	unique_ptr	shared_ptr	Implementation Mechanism
Acquire in Constructor	Takes raw pointer or make_unique	Takes raw pointer or make_shared	Constructor stores pointer, initializes control block
Release in Destructor	Calls delete (or custom deleter)	Decrements refcount, deletes if 0	Destructor invokes deleter when appropriate
Exception Safety	Destructor runs during stack unwinding	Destructor runs during stack unwinding	C++ guarantees destructor calls on scope exit
Copy Semantics	Deleted (exclusive ownership)	Increments reference count	Compiler-enforced or atomic operation
Move Semantics	Transfers ownership, nullifies source	Creates new owner, moves pointer	Efficient ownership transfer
Custom Deleters	Template parameter (zero overhead)	Type-erased in control block	Callable object invoked on deletion

Smart Pointers in Modern C++

Theory

📌 2. RAII and Automatic Resource Management (2/4)

RAII Benefits in Action

Problem with Manual Management	Smart Pointer Solution	Mechanism
Memory Leaks (forgot delete)	Automatic deletion in destructor	Compiler guarantees destructor call
Double Delete	unique_ptr prevents copies, shared_ptr uses refcount	Compile-time or runtime prevention
Dangling Pointers	Object deleted when last owner destroyed	Ownership tracking
Exception Leaks	Destructors run during stack unwinding	RAII guarantee
Ownership Confusion	Type encodes ownership semantics	unique vs shared vs weak
Resource Ordering	Destructors called in reverse construction order	LIFO stack unwinding

Smart Pointers in Modern C++

Theory

📌 2. RAII and Automatic Resource Management (3/4)

Code Example: RAII Exception Safety

```
#include <memory>
#include <iostream>
#include <stdexcept>

class Resource {
public:
    Resource(int id) : id_(id) {
        std::cout << "Resource " << id_ << " acquired\n";
    }
    ~Resource() {
        std::cout << "Resource " << id_ << " released\n";
    }
private:
    int id_;
};

// ❌ Manual management - exception unsafe
void manualApproach() {
    Resource* r1 = new Resource(1);
    Resource* r2 = new Resource(2);

    if (rand() % 2) {
        throw std::runtime_error("Error");  // ❌ Leak: r1 and r2 never deleted
    }

    delete r2;  // Never reached if exception thrown
    delete r1;
}

// ✅ Smart pointers - automatically exception-safe
void smartPointerApproach() {
    auto r1 = std::make_unique<Resource>(1);  // RAII object 1
    auto r2 = std::make_unique<Resource>(2);  // RAII object 2

    if (rand() % 2) {
        throw std::runtime_error("Error");  // ✅ OK: destructors run during unwinding
    }

    // Even if exception thrown, r2 then r1 automatically released (LIFO)
}
```

Smart Pointers in Modern C++

Theory

📌 2. RAII and Automatic Resource Management (4/4)

Stack Unwinding and Smart Pointers

When exceptions are thrown, C++ performs stack unwinding in reverse construction order (LIFO). Smart pointers leverage this guarantee:

```
void demonstrateStackUnwinding() {
    auto ptr1 = std::make_unique<Resource>(1);  // Constructed first
    auto ptr2 = std::make_unique<Resource>(2);  // Constructed second
    auto ptr3 = std::make_unique<Resource>(3);  // Constructed third

    throw std::runtime_error("Error");

    // Stack unwinding order:
    // 1. ptr3 destroyed (releases Resource 3)
    // 2. ptr2 destroyed (releases Resource 2)
    // 3. ptr1 destroyed (releases Resource 1)
    // All cleanup automatic - no manual try-catch needed
}
```

Smart Pointers in Modern C++

Theory

📌 3. Modern C++ Best Practices with Smart Pointers (1/6)

Modern C++ strongly favors smart pointers over manual memory management. Raw owning pointers (where the pointer is responsible for deletion) are considered a code smell.

The Smart Pointer Hierarchy of Preference

Priority	Choice	When to Use	Reasoning
1st	`std::unique_ptr`	Default for all dynamic allocation	Zero overhead, clear ownership, move-only
2nd	`std::shared_ptr`	Only when genuinely need shared ownership	Reference counting has cost
3rd	`std::weak_ptr`	Break cycles, caching, observation	Prevents leaks in circular structures
4th	Raw pointer/reference	Non-owning temporary access	For borrowing, not owning
Never	Raw owning pointer	Legacy code only	Leak-prone, exception-unsafe

Factory Function Patterns

Pattern	Return Type	When to Use	Example
Exclusive ownership	`unique_ptr<T>`	Single owner, caller takes ownership	`auto conn = createConnection();`
Shared ownership	`shared_ptr<T>`	Multiple owners expected	`auto cache = getSharedCache();`
Polymorphic factory	`unique_ptr<Base>`	Return derived via base pointer	`auto sensor = createSensor(type);`
Optional ownership	`unique_ptr<T>` or `nullptr`	May fail to create	`auto obj = tryCreate();`

Smart Pointers in Modern C++

Theory

📌 3. Modern C++ Best Practices with Smart Pointers (2/6)

Code Example: Factory Functions with Smart Pointers

```
// ✅ Best practice: Return unique_ptr for exclusive ownership
std::unique_ptr<Connection> createConnection(const std::string& url) {
    auto conn = std::make_unique<Connection>(url);
    conn->configure();
    return conn;  // Move, no copy
}

// ✅ Return shared_ptr when multiple owners genuinely needed
std::shared_ptr<Logger> getLogger() {
    static auto logger = std::make_shared<Logger>();  // Singleton pattern
    return logger;  // Multiple callers share same logger
}

// ✅ Polymorphic factory with unique_ptr
std::unique_ptr<Sensor> createSensor(SensorType type) {
    switch (type) {
        case SensorType::LIDAR:
            return std::make_unique<LidarSensor>();
        case SensorType::CAMERA:
            return std::make_unique<CameraSensor>();
    }
}

void usage() {
    auto conn = createConnection("http://api.example.com");
    // Clear ownership transfer, automatic cleanup

    auto logger = getLogger();
    auto logger2 = getLogger();  // Shares same instance

    auto sensor = createSensor(SensorType::LIDAR);
    // Caller owns sensor, can move or store
}
```

Smart Pointers in Modern C++

Theory

📌 3. Modern C++ Best Practices with Smart Pointers (3/6)

Function Parameter Guidelines

Scenario	Parameter Type	Rationale
Non-owning use	`T*` or `T&`	Most efficient, clear non-ownership
Take ownership (sink)	`unique_ptr<T>` by value	Explicit ownership transfer via move
Share ownership	`shared_ptr<T>` by value	Function becomes co-owner
Observe, don't own	`const shared_ptr<T>&`	No refcount changes, read-only
Optional parameter	`T* = nullptr`	Can be null, non-owning
Transfer or share	`unique_ptr<T>` or `shared_ptr<T>`	Caller decides via move or copy

📌 3. Modern C++ Best Practices with Smart Pointers (4/6)

Code Example: Parameter Conventions

```
// ✅ Non-owning use: raw pointer or reference
void processData(const Data* data) {
    if (data) {
        // Temporary use only, doesn't extend lifetime
    }
}

// ✅ Take ownership: unique_ptr by value
void storeConnection(std::unique_ptr<Connection> conn) {
    connections_.push_back(std::move(conn));  // Takes ownership
}

// ✅ Share ownership: shared_ptr by value
void registerCallback(std::shared_ptr<Callback> callback) {
    callbacks_.push_back(callback);  // Shares ownership
}

// ✅ Observe without owning: const reference
void logStats(const std::shared_ptr<Stats>& stats) {
    std::cout << stats->count();  // No refcount change
}

// ❌ Wrong: shared_ptr by value when not needed
void wrongApproach(std::shared_ptr<Data> data) {  // Unnecessary atomic ops
    data->process();  // Just using, not storing
}
```

Smart Pointers and the Rule of Zero

When all resources are managed by smart pointers and standard containers, classes follow the **Rule of Zero**: no custom destructor, copy constructor, move constructor, copy assignment, or move assignment needed.

Smart Pointers in Modern C++

Theory

📌 3. Modern C++ Best Practices with Smart Pointers (5/6)

Rule of Zero with Smart Pointers

Without Smart Pointers (Rule of Five)	With Smart Pointers (Rule of Zero)
Manual destructor `delete ptr_;`	Compiler-generated destructor works
Custom copy constructor (deep copy)	Compiler-generated copy works
Custom copy assignment	Compiler-generated assignment works
Custom move constructor	Compiler-generated move works
Custom move assignment	Compiler-generated move assignment works
Prone to bugs (forgot one? resource leak)	Cannot have bugs (nothing to forget)

Code Example: Rule of Zero

```
// ❌ Manual management: Rule of Five required
class ManualWidget {
    Connection* conn_;
    Data* data_;
public:
    ManualWidget() : conn_(new Connection()), data_(new Data()) {}
    ~ManualWidget() { delete data_; delete conn_; }
    // Need custom copy constructor, copy assignment, move constructor, move assignment
    // Forgetting any one = bugs
};

// ✅ Smart pointers: Rule of Zero
class ModernWidget {
    std::unique_ptr<Connection> conn_ = std::make_unique<Connection>();
    std::unique_ptr<Data> data_ = std::make_unique<Data>();
    // Compiler generates all special members correctly
    // Move-only due to unique_ptr (safe by default)
    // Zero manual memory management
};
```

Smart Pointers in Modern C++

Theory

📌 3. Modern C++ Best Practices with Smart Pointers (6/6)

Key Takeaway: Modern C++ eliminates manual memory management through smart pointers. Default to `unique_ptr`, upgrade to `shared_ptr` only when genuinely needed, use `weak_ptr` to break cycles. Raw pointers are for non-owning temporary access only, never for ownership. Smart pointers enable the Rule of Zero, making code safer, more maintainable, and exception-safe by default.

Smart Pointers in Modern C++

Edge Cases

📌 1. Circular References with shared_ptr

When two or more objects hold shared_ptrs to each other, they create a reference cycle where reference counts never reach zero, causing a memory leak. This commonly occurs in bidirectional relationships like parent-child hierarchies, doubly-linked lists, observer patterns, and graph structures.

```
struct Node {
    std::shared_ptr<Node> next;
    ~Node() { std::cout << "Node destroyed\n"; }
};

void createLeak() {
    auto a = std::make_shared<Node>();
    auto b = std::make_shared<Node>();

    a->next = b;  // b's use_count becomes 2
    b->next = a;  // a's use_count becomes 2

    // When a and b go out of scope:
    // - a's destructor releases its shared_ptr to b
    // - b's destructor would release its shared_ptr to a
    // - But destructors never run because use_count stays at 1
    // ❌ Memory leak: neither object is ever deleted
}
```

The cycle prevention requires replacing one direction with weak_ptr, breaking the strong ownership loop. The choice of which direction should be weak depends on the natural ownership relationship—typically back-pointers or parent-references become weak_ptrs.

Smart Pointers in Modern C++

Edge Cases

📌 2. Creating shared_ptr from unique_ptr

Unique_ptr can be implicitly converted to shared_ptr through move semantics, providing a convenient upgrade path when ownership needs change. The conversion transfers ownership from the unique_ptr to a new shared_ptr with initial reference count of 1.

```
std::unique_ptr<Widget> unique = std::make_unique<Widget>();
std::shared_ptr<Widget> shared = std::move(unique);  // ✅ Ownership transferred
// unique is now nullptr

// ❌ Cannot go backwards:
// std::unique_ptr<Widget> unique2 = shared;  // Compile error
```

This one-way conversion reflects the ownership semantics: exclusive ownership can become shared, but shared ownership cannot become exclusive without explicit coordination (checking use_count() == 1).

Smart Pointers in Modern C++

Edge Cases

📌 3. Polymorphic Deletion Without Virtual Destructor

Smart pointers correctly support polymorphic deletion only when the base class has a virtual destructor. Without it, deleting through a base pointer causes undefined behavior by only calling the base destructor.

```
struct Base {
    ~Base() { std::cout << "Base destroyed\n"; }  // ❌ Not virtual
};

struct Derived : Base {
    int* data = new int[100];
    ~Derived() {
        delete[] data;
        std::cout << "Derived destroyed\n";
    }
};

void leak() {
    std::unique_ptr<Base> ptr = std::make_unique<Derived>();
    // ❌ When ptr destroyed, only ~Base() called
    // Derived's destructor never runs → data leaks
}

void correct() {
    struct Base { virtual ~Base() = default; };  // ✅ Virtual destructor
    std::unique_ptr<Base> ptr = std::make_unique<Derived>();
    // ✅ Correct polymorphic deletion
}
```

This is one of the few cases where smart pointers don't automatically solve memory problems—the class hierarchy must be correctly designed with virtual destructors.

Smart Pointers in Modern C++

Edge Cases

📌 4. Aliasing Constructor of shared_ptr

The shared_ptr aliasing constructor allows creating a shared_ptr that participates in reference counting for one object but provides access to a different object. This enables managing an object's lifetime while exposing a sub-object.

```
struct Widget {
    int x, y;
};

auto widget = std::make_shared<Widget>();
std::shared_ptr<int> xptr(widget, &widget->x);  // Aliasing constructor

// xptr points to x but keeps entire Widget alive
// Useful for: exposing members, arrays, pimpl patterns
```

The aliasing constructor takes two parameters: a shared_ptr to keep alive and a pointer to expose. When the last aliasing or original shared_ptr is destroyed, the original object is deleted.

Smart Pointers in Modern C++

Edge Cases

📌 5. Double Ownership Through get()

The get() method returns the raw pointer without transferring ownership or affecting reference count. Creating a second smart pointer from this raw pointer creates double ownership and causes double deletion.

```
auto ptr1 = std::make_unique<int>(42);
int* raw = ptr1.get();  // ✅ Safe: borrow only

std::unique_ptr<int> ptr2(raw);  // ❌ Double ownership!
// When ptr1 and ptr2 destroyed → double delete → crash
```

The get() method should only be used for borrowing when interfacing with APIs that require raw pointers. Never create a second owning smart pointer from the result of get().

Smart Pointers in Modern C++

Edge Cases

📌 6. weak_ptr Promotion Race Condition

Between checking expired() and calling lock(), the object can be destroyed by another thread. Always use lock() directly rather than checking expired() first.

```
std::weak_ptr<Widget> weak;

// ❌ Race condition:
if (!weak.expired()) {
    auto shared = weak.lock();  // Might be null if destroyed between checks
    shared->use();              // ❌ Potential null dereference
}

// ✅ Correct:
if (auto shared = weak.lock()) {
    shared->use();  // Safe: object kept alive during this scope
}
```

The lock() method atomically checks validity and increments the reference count, eliminating the race condition window.

Smart Pointers in Modern C++

Edge Cases

📌 7. make_shared Exception Safety vs Custom Allocator

While make_shared provides exception safety and performance benefits, it prevents custom allocators and can delay memory reclamation when weak_ptrs outlive all shared_ptrs.

```
// make_shared allocates object + control block together
auto p1 = std::make_shared<Large>();  // One allocation

// Separate allocations allow independent lifetimes
std::shared_ptr<Large> p2(new Large());  // Two allocations

// With make_shared, if weak_ptrs exist, memory not freed
// until weak_ptrs destroyed, even though object was deleted
```

For very large objects or when precise memory control is needed, the separate allocation pattern may be preferable despite slightly lower performance.

Smart Pointers in Modern C++

Code Examples

📌 1. Basic unique_ptr Ownership Transfer

Unique_ptr enforces exclusive ownership through deleted copy constructor and copy assignment operator. Ownership can only be transferred using std::move, which leaves the source pointer in a null state. This compile-time enforcement prevents accidental double ownership.

```
#include <memory>
#include <iostream>

struct Resource {
    int id;
    Resource(int i) : id(i) { std::cout << "Resource " << id << " created\n"; }
    ~Resource() { std::cout << "Resource " << id << " destroyed\n"; }
};

int main() {
    std::unique_ptr<Resource> ptr1 = std::make_unique<Resource>(1);

    // std::unique_ptr<Resource> ptr2 = ptr1;  // ❌ Copy deleted
    std::unique_ptr<Resource> ptr2 = std::move(ptr1);  // ✅ Ownership transfer

    if (!ptr1) {
        std::cout << "ptr1 is now null\n";
    }

    std::cout << "ptr2 owns resource: " << ptr2->id << "\n";
}  // Resource destroyed when ptr2 goes out of scope
```

Smart Pointers in Modern C++

Code Examples

📌 2. shared_ptr Reference Counting

The shared_ptr maintains a reference count in its control block, atomically incremented on copy and decremented on destruction. When the count reaches zero, the managed object is deleted. Multiple shared_ptrs can safely own the same object across different scopes and threads.

```
#include <memory>
#include <iostream>

class Widget {
public:
    Widget(const std::string& n) : name(n) {
        std::cout << "Widget " << name << " constructed\n";
    }
    ~Widget() {
        std::cout << "Widget " << name << " destroyed\n";
    }
private:
    std::string name;
};

void demonstrate() {
    std::shared_ptr<Widget> sp1 = std::make_shared<Widget>("A");
    std::cout << "Count: " << sp1.use_count() << "\n";  // 1

    {
        std::shared_ptr<Widget> sp2 = sp1;  // ✅ Copy increments count
        std::shared_ptr<Widget> sp3 = sp1;
        std::cout << "Count: " << sp1.use_count() << "\n";  // 3
    }  // sp2 and sp3 destroyed, count decremented

    std::cout << "Count: " << sp1.use_count() << "\n";  // 1
}  // sp1 destroyed, count reaches 0, Widget deleted
```

Smart Pointers in Modern C++

Code Examples

📌 3. Breaking Circular References with weak_ptr

Using weak_ptr for the backward reference breaks the circular dependency. When a and b go out of scope, a can be destroyed (count reaches 0), which releases its shared_ptr to b, allowing b to be destroyed. The weak_ptr prevents the ownership cycle that would leak memory.

```
#include <memory>
#include <iostream>
#include <string>

struct Node {
    std::string name;
    std::shared_ptr<Node> next;      // Strong reference forward
    std::weak_ptr<Node> prev;        // Weak reference backward

    Node(const std::string& n) : name(n) {
        std::cout << "Node " << name << " created\n";
    }
    ~Node() {
        std::cout << "Node " << name << " destroyed\n";
    }
};

int main() {
    auto a = std::make_shared<Node>("A");
    auto b = std::make_shared<Node>("B");

    a->next = b;        // a owns b: b.use_count = 2
    b->prev = a;        // b observes a: a.use_count = 1

    std::cout << "a count: " << a.use_count() << "\n";  // 1
    std::cout << "b count: " << b.use_count() << "\n";  // 2

    // Access prev through weak_ptr
    if (auto prevNode = b->prev.lock()) {
        std::cout << "b's prev is: " << prevNode->name << "\n";
    }
}  // Both nodes properly destroyed: no cycle!
```

Smart Pointers in Modern C++

Code Examples

📌 4. Custom Deleters for Non-Memory Resources (1/2)

Custom deleters enable smart pointers to manage any resource type, not just heap memory. For unique_ptr, the deleter type is part of the template parameters, maintaining zero overhead. For shared_ptr, the deleter is type-erased in the control block, adding slight overhead but providing type flexibility.

```
#include <memory>
#include <cstdio>
#include <iostream>

// Custom deleter for FILE*
struct FileCloser {
    void operator()(FILE* f) const {
        if (f) {
            std::cout << "Closing file\n";
            fclose(f);
        }
    }
};

// Using lambda with unique_ptr
void useUniquePtr() {
    auto closer = [](FILE* f) {
        if (f) {
            std::cout << "Lambda closing file\n";
            fclose(f);
        }
    };

    std::unique_ptr<FILE, decltype(closer)> file(
        fopen("test.txt", "w"),
        closer
    );

    if (file) {
        fprintf(file.get(), "Hello World\n");
    }
}  // File automatically closed via custom deleter

// Using lambda with shared_ptr
void useSharedPtr() {
// ...continues on next slide
```

Smart Pointers in Modern C++

Code Examples

📌 4. Custom Deleters for Non-Memory Resources (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    std::shared_ptr<FILE> file(
        fopen("test.txt", "r"),
        [](FILE* f) {
            if (f) {
                std::cout << "Shared_ptr closing file\n";
                fclose(f);
            }
        }
    );
}  // File automatically closed
```

Smart Pointers in Modern C++

Code Examples

📌 5. Managing Arrays with Smart Pointers (1/2)

The unique_ptr has a partial specialization for arrays (unique_ptr) that correctly uses delete[] instead of delete. Shared_ptr gained similar support in C++17. However, using std::vector wrapped in a smart pointer is generally preferable, providing bounds checking, size tracking, and resizability.

```
#include <memory>
#include <iostream>

void uniquePtrArray() {
    // ✅ Correct: unique_ptr<T[]> for arrays
    std::unique_ptr<int[]> arr(new int[10]);

    // Array access syntax works
    for (int i = 0; i < 10; ++i) {
        arr[i] = i * 10;
    }

    std::cout << "arr[5] = " << arr[5] << "\n";  // 50
}  // Correctly calls delete[]

void sharedPtrArray() {
    // Pre-C++17: Need custom deleter
    std::shared_ptr<int> arr(
        new int[10],
        [](int* p) { delete[] p; }  // ✅ Custom deleter
    );

    // ❌ No array syntax: arr[i] doesn't work
    // Must use pointer arithmetic

    // ✅ Better: Use vector
    auto vec = std::make_shared<std::vector<int>>(10);
    (*vec)[5] = 50;
}

void avoidArrayIssues() {
    // ✅ Best practice: Use vector
    std::unique_ptr<std::vector<int>> vec =
        std::make_unique<std::vector<int>>(10);

// ...continues on next slide
```

Smart Pointers in Modern C++

Code Examples

📌 5. Managing Arrays with Smart Pointers (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    (*vec)[5] = 50;
    vec->push_back(60);  // Resizable
}
```

Smart Pointers in Modern C++

Code Examples

📌 6. Factory Functions Returning Ownership

Factory functions returning smart pointers clearly communicate ownership transfer. Returning unique_ptr is preferred as it's more restrictive—callers can always convert to shared_ptr if needed, but not vice versa. The return value optimization (RVO) and move semantics make this pattern efficient with no unnecessary copies.

```
#include <memory>
#include <string>

class Connection {
    std::string url;
public:
    Connection(const std::string& u) : url(u) {}
    void send(const std::string& data) {}
};

// ✅ Return unique_ptr for exclusive ownership transfer
std::unique_ptr<Connection> createConnection(const std::string& url) {
    auto conn = std::make_unique<Connection>(url);
    // Setup, configuration...
    return conn;  // Move, no copy
}

// ✅ Return shared_ptr when multiple owners expected
std::shared_ptr<Connection> createSharedConnection(const std::string& url) {
    return std::make_shared<Connection>(url);
}

void use() {
    auto conn1 = createConnection("http://api.example.com");
    conn1->send("data");
    // Clear ownership transfer, automatic cleanup

    auto conn2 = createSharedConnection("http://api.example.com");
    auto conn3 = conn2;  // Multiple owners
}
```

Smart Pointers in Modern C++

Code Examples

📌 7. Using weak_ptr for Caching (1/2)

Using weak_ptr for caching allows objects to be cached without preventing their destruction. When no external shared_ptrs exist, the object is automatically cleaned up. Subsequent cache accesses detect the expired weak_ptr and create a fresh object, providing automatic cache invalidation.

```
#include <memory>
#include <map>
#include <string>
#include <iostream>

class ExpensiveObject {
public:
    ExpensiveObject(const std::string& id) : id_(id) {
        std::cout << "Creating expensive object: " << id_ << "\n";
    }
    ~ExpensiveObject() {
        std::cout << "Destroying expensive object: " << id_ << "\n";
    }
private:
    std::string id_;
};

class ObjectCache {
    std::map<std::string, std::weak_ptr<ExpensiveObject>> cache_;

public:
    std::shared_ptr<ExpensiveObject> get(const std::string& id) {
        // Try to get from cache
        auto it = cache_.find(id);
        if (it != cache_.end()) {
            if (auto cached = it->second.lock()) {
                std::cout << "Cache hit for: " << id << "\n";
                return cached;  // Return existing object
            }
        }

        // Create new object
        std::cout << "Cache miss for: " << id << "\n";
        auto obj = std::make_shared<ExpensiveObject>(id);
        cache_[id] = obj;  // Store weak_ptr
// ...continues on next slide
```

Smart Pointers in Modern C++

Code Examples

📌 7. Using weak_ptr for Caching (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        return obj;
    }
};

void demonstrateCache() {
    ObjectCache cache;

    {
        auto obj1 = cache.get("item1");  // Cache miss, creates
        auto obj2 = cache.get("item1");  // Cache hit, reuses
    }  // Objects destroyed when last shared_ptr released

    auto obj3 = cache.get("item1");  // Cache miss, creates new
}
```

Smart Pointers in Modern C++

Code Examples

📌 8. Exception Safety with Smart Pointers (1/2)

Smart pointers provide automatic exception safety through RAII. When exceptions occur, stack unwinding invokes destructors of all stack-allocated objects, including smart pointers, which then properly clean up their managed resources. This eliminates the need for try-catch blocks purely for resource cleanup.

```
#include <memory>
#include <stdexcept>
#include <iostream>

class Resource {
public:
    Resource() { std::cout << "Resource acquired\n"; }
    ~Resource() { std::cout << "Resource released\n"; }
};

void riskyOperation() {
    if (rand() % 2) {
        throw std::runtime_error("Random failure");
    }
}

// ❌ Not exception-safe
void rawPointerVersion() {
    Resource* r = new Resource();
    try {
        riskyOperation();  // May throw
        delete r;
    } catch(...) {
        delete r;  // Must duplicate cleanup
        throw;
    }
}

// ✅ Exception-safe with smart pointer
void smartPointerVersion() {
    std::unique_ptr<Resource> r = std::make_unique<Resource>();
    riskyOperation();  // If throws, destructor still runs
    // No explicit cleanup needed
}

// ...continues on next slide
```

Smart Pointers in Modern C++

Code Examples

📌 8. Exception Safety with Smart Pointers (2/2)

[Code continues across slides]

```
// ...continued from previous slide
// ✅ Exception-safe in function parameters
void processResources(
    std::unique_ptr<Resource> r1,
    std::unique_ptr<Resource> r2
) {
    riskyOperation();
}

void callProcess() {
    // ✅ Exception-safe: make_unique guarantees no leaks
    processResources(
        std::make_unique<Resource>(),
        std::make_unique<Resource>()
    );
}
```

What You Learned

Key Takeaways

- ✓ Smart Pointer Types and Ownership Models
- ✓ RAII and Automatic Resource Management
- ✓ Modern C++ Best Practices with Smart Pointers
- ✓ Circular References with shared_ptr

→ Swipe to learn →